
Software Cost Estimation

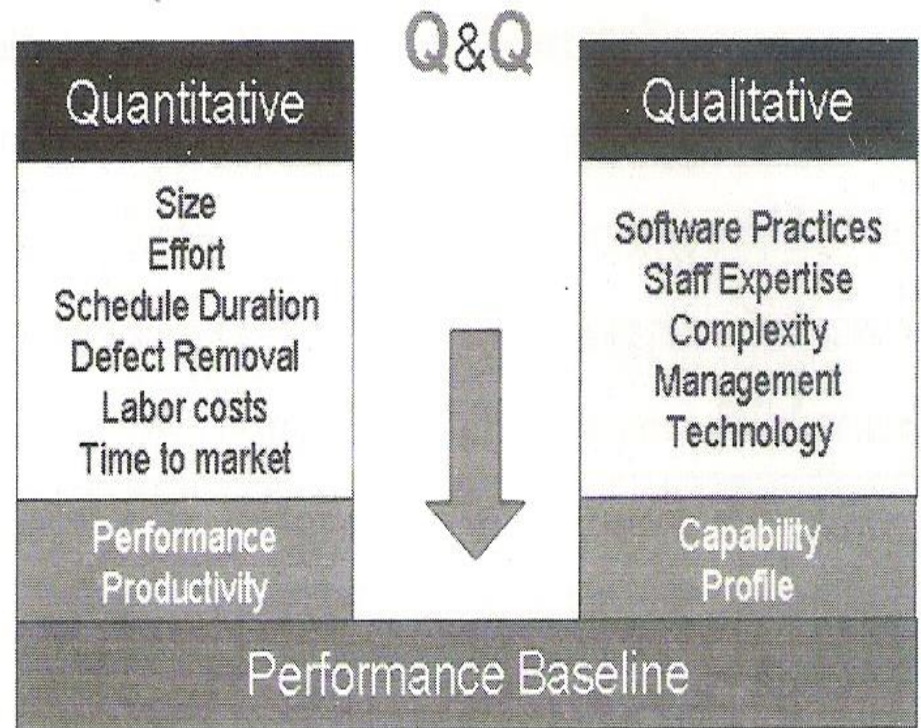
อ.วิวัฒน์ ชินนาทศิริกุล

Software Estimation

- การประมาณการซอฟต์แวร์ เป็นส่วนที่สำคัญในการวางแผนงาน เนื่องจากแผนงานนั้นจะอยู่บนพื้นฐานของสิ่งที่ต้องการทำการจัดสร้างหรือพัฒนา โดยในส่วนของซอฟต์แวร์นั้นมุมมองหลักที่มองถึง คือ
 1. ขนาด (Size)
 2. ค่าใช้จ่าย (Cost)
 3. บุคลากรที่ใช้ในการพัฒนา (Effort)

การประมาณการขนาดซอฟต์แวร์ (Size Estimation)

- สิ่งแรกที่จะต้องทำก่อนการเริ่มต้นการประมาณการ คือ การวัด แยก ลักษณะการวัดออกเป็น 2 ประเภท คือ
 1. การวัดในเชิงปริมาณ (Software Quantitative)
 2. การวัดเชิงคุณภาพ (Software Qualitative)



* From DAvid Garmus, David Herron, Function Point Analysis, p.5

การประมาณการขนาดซอฟต์แวร์ (ต่อ)

- กรรมวิธีที่ใช้ในการวัดขนาดของซอฟต์แวร์ มี 2 ลักษณะ คือ
 1. การใช้ Model ในการประเมินราคาซอฟต์แวร์
 2. Function Point (FP)

การใช้ Model ในการประเมินราคาซอฟต์แวร์

1. วิธีของ LaBolle ได้เสนอวิธีที่เน้นเทคนิคทางคณิตศาสตร์

การประเมินราคาจะต้องพิจารณาถึงวิธีการในการคำนวณอย่างน้อย

1 วิธีการดังนี้

- ราคาต่อหน่วย เช่น ราคาต่อคำสั่ง ต่อโปรแกรมย่อย ต่อชุดคำสั่ง หรือ ต่อโมดูล หรือ ราคาต่อระบบ และ ราคาต่อการทำกิจกรรมอย่างหนึ่ง
- เปอร์เซนต์จากราคารวม เช่น ให้ราคาโปรแกรมคอมพิวเตอร์เป็น $x\%$ ของค่าพัฒนาทั้งหมด
- การเปรียบเทียบจำเพาะ เช่น ความคล้ายคลึงของโปรแกรมที่ต้องการพัฒนาใหม่กับโปรแกรมที่มีอยู่แล้ว เช่น อาจเพิ่มความต้องการ ในโปรแกรมใหม่ ดังนั้นราคาจะเท่ากับ ราคาของโปรแกรมเก่า บวกกับราคาของการพัฒนาความต้องการ

สูตรที่ใช้คำนวณ

$$\text{Cost , } C = K_1X + K_2Y + K_3Z$$

X = จำนวนหน้าจอ Y = จำนวนชิ้นโปรแกรม Z = จำนวนข้อมูล

K_1, K_2, K_3 = เป็นค่าคงที่ที่กำหนดค่าไว้

2. Wolverton ได้เสนอเทคนิคการจัดโมเดลในการประเมินราคาซอฟต์แวร์ไว้ดังนี้

2.1 การประเมินแบบ Top-down

- ผู้ประเมินจะยึดเอาราคาทั้งหมดของงานส่วนใหญ่ในโครงการที่ทำมาก่อนเป็นหลัก และด้วยประวัติที่ผ่านมาพิจารณาประกอบกับการแสดงความคิดเห็นก็จะสามารถประเมินราคาใหม่ได้

2.2 การประเมินราคาโดยพิจารณาจากความเหมือนและความแตกต่าง

- ผู้ประเมินจะแบ่งงานการประเมินโดยคำนึงถึงรายละเอียดของโปรแกรมใหม่ ที่พัฒนาขึ้น โดยเปรียบเทียบความเหมือนและความแตกต่างกับโปรแกรมในโครงการเก่า สำหรับงานที่ไม่สามารถเปรียบเทียบได้ก็จะใช้วิธีอื่น

2.3 กระประเมินโดยใช้วิธี Bottom-up

- วิธีนี้ที่ใช้กันอย่างกว้างขวางในหน่วยงานการประเมินของรัฐบาล โดยงานทั้งหมดจะถูกแบ่งออกเป็นชิ้นย่อย ๆ และจะแบ่งให้เล็กลงเรื่อย ๆ จนถึงขั้นที่สามารถสร้างได้อย่างชัดเจน จากนั้นจึงทำการประเมินราคาของแต่ละผลงานและนำมารวมกันเป็นราคาของโครงการทั้งหมด

3. Walston และ Felix ได้เริ่มเก็บข้อมูลตั้งแต่ปี ค.ศ. 1973 และในปี 1977 สามารถสรุปได้ในรูปแบบสมการดังนี้คือ

$$E = 5.2(KDSI)^{0.91}$$

$E = \text{Effort}$ มีหน่วยเป็น คน-เดือน

$KDSI = \text{Kilo}(1000 \text{ บรรทัด}) \text{ of Delivered Source Instruction}$

■ นิยามของ DSI

- ❑ นับเฉพาะบรรทัดที่มีการจัดสั่งเป็น **Source Code** ไม่นับรวมส่วนของการทดสอบ หรือ ส่วนงานที่รองรับการทำงานอื่นๆ
- ❑ นับเฉพาะบรรทัดที่พัฒนาโดยบุคลากร ไม่นับรวมสิ่งที่ระบบงานสามารถ สร้างขึ้นมาได้อัตโนมัติ
- ❑ ถือว่า 1 คำสั่งคือ 1 **Line of Code**
- ❑ นับส่วนของการประกาศค่า (**Declaration**) เป็นส่วนของ **Instruction**
- ❑ ไม่นับในส่วนขยายความ หรือ **Comment**

4. Boehm B.W. ได้พัฒนาโมเดล COCOMO (Constructive Cost Model) ขึ้นในปี ค.ศ. 1981 โดยใช้การวิเคราะห์ข้อมูลจาก 63 โครงการ วิธีการของ COCOMO นี้เป็นวิธีการวัด Effort ในการพัฒนาซอฟต์แวร์ที่คิดเป็น คน-เดือน โดยได้กำหนดปัจจัยที่มีผลต่อราคาซอฟต์แวร์ได้ 4 อย่างกว้างๆ คือ ผลผลิตคอมพิวเตอร์ บุคลากร โครงการ โดยรูปแบบของโมเดลจะแบ่งออกเป็น 3 ลักษณะด้วยกันคือ

- ❑ Basic COCOMO Model : เป็นโมเดลที่มีการกำหนดค่าคงที่ค่าเดียวเพื่อการคำนวณในการพัฒนาซอฟต์แวร์เป็นขนาดโปรแกรมซึ่งปรากฏในรูปแบบ Lines of Code (LOC)
- ❑ Intermediate COCOMO Model : คำนวณ Effort ในการพัฒนาซอฟต์แวร์เป็นขนาดของโปรแกรมและรวมปัจจัยที่มีผลกระทบต่อราคา
- ❑ Advanced COCOMO Model : จะรวมปัจจัยที่มีผลกระทบต่อราคาทั้งหมดในทุกๆขั้นตอน เช่น ในกรณีการวิเคราะห์ และ การออกแบบ

Function Point

- เป็นการวัดขนาดของซอฟต์แวร์ด้วยการนับจำนวนฟังก์ชันการทำงานของโปรแกรม การวัดด้วยฟังก์ชันพอยต์ จะมุ่งเน้นที่การวัดด้วยฟังก์ชันหรือการวัดโดยผ่านมุมมองความต้องการของซอฟต์แวร์
- ช่วยลดปัญหาด้านความแตกต่างของภาษาโปรแกรมมิ่งที่ใช้
- Allan Albrecht [1] John Gaffney, Jr [2] ได้ออกแบบ FPs ที่ใช้วัดฟังก์ชันพอยต์ FPs เป็นผลรวมของขนาด ข้อมูลเข้า, ข้อมูลออก, ข้อมูลความต้องการ, เพิ่มข้อมูล และส่วนของโปรแกรมที่ใช้ในการติดต่อกับลูกค้า

กระบวนการนับฟังก์ชันพอยต์ มีลักษณะดังนี้

1. นำ Requirement ที่เก็บรวบรวมไว้มาทำการแบ่งฟังก์ชันพอยต์
2. ประเมินความซับซ้อนของฟังก์ชัน
3. เปรียบเทียบความซับซ้อน เพื่อให้ได้ระดับความซับซ้อน เพื่อคำนวณฟังก์ชันพอยต์ที่ยังไม่ได้ปรับค่า (Unadjusted Function Point : UFP)
4. คำนวณค่าตัวแปรปรับค่า (Value Adjustment Factor) ตามลักษณะของโครงการ
5. คำนวณจำนวนฟังก์ชันพอยต์ที่ผ่านการปรับค่า (Adjusted Function Point : AFP)
6. ฟังก์ชันพอยต์ที่ผ่านการปรับค่า สามารถนำไปคำนวณเป็น LOC ได้

สูตรคำนวณ Function Point : FP

$$FP = UFP * VAF$$

- FP คือ ขนาดของซอฟต์แวร์
- UFP (Unadjusted Function Point) ค่า FP ที่ยังไม่ได้ถูกปรับแต่ง
- VAF (Value Adjustment Factor) ค่าปัจจัยคุณลักษณะของระบบ

การคำนวณหา FP ที่ยังไม่ได้ปรับแต่ง

- ประเภทของฟังก์ชันพอยต์ สามารถแบ่งได้ 5 ลักษณะหลัก คือ
 - ❑ External Input (EI)
 - ❑ External Output (EO)
 - ❑ External Inquiry (EQ)
 - ❑ Internal Logical Files (ILF)
 - ❑ External Interface Files (EIF)

การคำนวณหา FP ที่ยังไม่ได้ปรับแต่ง (UFP)

ฟังก์ชัน	รายละเอียด
External Inputs (EI)	ข้อมูลที่รับเข้ามาในระบบ (อาจเป็นข้อมูลทางธุรกิจหรือข้อมูลควบคุม) เพื่อนำไปอัปเดตข้อมูลใน ILF เช่น ข้อมูลในกระบวนการ เพิ่ม ลบ แก้ไข ข้อมูล เป็นต้น
External Outputs (EO)	ข้อมูลที่เป็นผลลัพธ์จากการประมวลผลข้อมูลที่ได้รับจากภายในระบบ ให้นำมาแสดงผลข้อมูลที่มีรูปแบบแตกต่างกัน
External Queries (EQ)	กระบวนการดึงข้อมูลและประมวลผลเพื่อแสดงผลต่อผู้ใช้ (คือ Query ข้อมูลนั่นเอง)
Internal Logical Files (ILF)	ไฟล์ที่เกี่ยวข้องกับข้อมูลที่อยู่ในระบบตลอดช่วงอายุของระบบ และเป็นไฟล์ที่มักจะถูกบำรุงรักษาหรือปรับปรุงด้วยข้อมูลที่ได้รับจากภายนอก (EI) ให้นำมารวมเรคคอร์ดที่ทำหน้าที่เทียบเท่ากับไฟล์ด้วย
External Interface Files (EIF)	ไฟล์ที่เกี่ยวข้องกับข้อมูลที่ใช้เพื่อการอ้างอิงเท่านั้น และใช้ร่วมกับระบบอื่น EIF เป็นไฟล์ที่ถูกเรียกใช้โดยระบบที่จะพัฒนา แต่จะบำรุงรักษาหรือถูกสร้างโดยระบบอื่น

- ฟังก์ชันแต่ละประเภทเกิดจากการทำรายการข้อมูล (Transaction) ของผู้ใช้ จึงมีความซับซ้อนแตกต่างกันขึ้นอยู่กับสิ่งต่อไปนี้
 - จำนวนของข้อมูล ฟิลด์ข้อมูล (Data Element Type : DET)
 - จำนวนเรคคอร์ด (Record Element Type: RET)
 - ไฟล์ที่เกี่ยวข้อง (File Type Reference: FTR)

- ดังนั้นการนับฟังก์ชันแต่ละประเภท จึงต้องนับที่จำนวนของ DET , RET และ FTR ที่เกี่ยวข้องกับฟังก์ชันแต่ละประเภท แล้วนำมาเทียบกับตารางเกณฑ์ระดับความซับซ้อนของฟังก์ชัน
- เกณฑ์ความซับซ้อนแบ่งเป็น
 - ระดับต่ำ (Low)
 - ระดับปานกลาง (Average)
 - ระดับสูง (High)

ตารางเกณฑ์ระดับความซับซ้อนของฟังก์ชัน

FP Complexity Level			
Internal Logical Files and External Interface files			
Record Elements	Data Elements		
	1-19	20-50	51+
1	Low	Low	Average
2-5	Low	Average	High
6+	Average	High	High
Internal External Output and External Inquiry			
File Types	Data Elements		
	1-5	6-19	20+
0 or 1	Low	Low	Average
2-3	Low	Average	High
4+	Average	High	High
For External Input			
File Types	Data Elements		
	1-4	5-15	16+
0 or 1	Low	Low	Average
2-3	Low	Average	High
3+	Average	High	High

- จากตารางข้างบน จะได้ระดับความซับซ้อนของการทำงาน จากนั้นนำค่าความซับซ้อนที่เป็นค่าเฉลี่ยมาทำการคำนวณค่า Complexity weight ตามตารางถ่วงน้ำหนักดังนี้

Complexity-Weight			
Function Type	Complexity-Weight		
	Low	Average	High
1. Internal Logical File (ILF)	7	10	15
2. External Interface Files (EIF)	5	7	10
3. External Input (EI)	3	4	6
4. External Output (EO)	4	5	7
5. External Inquiry (EQ)	3	4	6

ตัวอย่าง การหาค่า UFP

- ถ้าข้อมูลสินค้าที่จะนำเข้าสู่ระบบ (EI) เกี่ยวข้องกับไฟล์ 2 ชนิด (FTR) และข้อมูลสินค้านี้ประกอบด้วยฟิลด์ข้อมูลไม่เกิน 15 ฟิลด์(DET)
 - เมื่อเทียบกับตารางเกณฑ์ระดับความซับซ้อนของฟังก์ชัน พบว่า EI มีระดับความซับซ้อนอยู่ที่ Average
 - เมื่อดูจากตารางถ่วงน้ำหนักแล้ว EI ที่มีค่าความซับซ้อนอยู่ที่ Average จะมีตัวคูณถ่วงน้ำหนักอยู่ที่ 4
 - จะได้ $UFP = 2 * 4 = 8$

การคำนวณค่าปัจจัยคุณลักษณะของระบบ VAF

การประเมิน VAF นั้นจะประเมินค่าของ 14 ปัจจัย ดังนี้

1. การติดต่อสื่อสารข้อมูล (Data Communication)
2. การประมวลผลข้อมูลแบบกระจาย (Distributed Data Processing)
3. ประสิทธิภาพของระบบ (Performance)
4. การแก้ไขค่าของระบบ (Configuration)
5. ปริมาณรายการข้อมูล (Transaction)
6. การป้อนข้อมูลเข้าสู่ระบบแบบออนไลน์ (Online Data Entry)

-
7. ประสิทธิภาพการใช้งานของผู้ใช้ (End user Efficiency)
 8. การปรับปรุงข้อมูลแบบออนไลน์ (Online Update)
 9. ความซับซ้อนของการประมวลผล (Complex Processing)
 10. การนำไปใช้ซ้ำได้ (Reusability)
 11. ความง่ายในการติดตั้ง (Installation Ease)
 12. ความง่ายในการดำเนินงาน (Operational Ease)
 13. การใช้งานได้หลายไซต์ (Multiple Sites)
 14. รองรับการเปลี่ยนแปลงความต้องการของผู้ใช้ (Change Requirement)
-

การประเมินนั้น แบ่งออกเป็น 6 ระดับ (0-5)

ระดับ		ความหมาย
0	Not Present	ไม่มีผลเกี่ยวข้องกับตัวแปรนั้น ๆ
1	Incidental Influence	มีความเกี่ยวข้องกับตัวแปรนั้น ๆ โดยมีเกิดขึ้นในระบบงาน ไม่กระทบต่อการทำงาน
2	Moderate Influence	มีความเกี่ยวข้องกับตัวแปรนั้น ๆ โดยมีเกิดขึ้นในระบบงาน กระทบต่อการทำงาน โดยทำให้การทำงานซับซ้อนขึ้นบ้างเล็กน้อย
3	Average Influence	มีความเกี่ยวข้องกับตัวแปรนั้น ๆ โดยมีเกิดขึ้นในระบบงาน กระทบต่อการทำงาน โดยทำให้การทำงานซับซ้อนขึ้น
4	Significant Influence	มีความเกี่ยวข้องกับตัวแปรนั้น ๆ โดยมีเกิดขึ้นในระบบงาน กระทบต่อการทำงาน โดยทำให้การทำงานซับซ้อนค่อนข้างมาก
5	Strong Influence	มีความเกี่ยวข้องกับตัวแปรนั้น ๆ โดยมีเกิดขึ้นในระบบงาน กระทบต่อการทำงาน โดยทำให้การทำงานซับซ้อนมาก

สูตรการคำนวณ VAF

$$\text{VAF} = 0.65 + [0.01 \times \text{Total DI}]$$

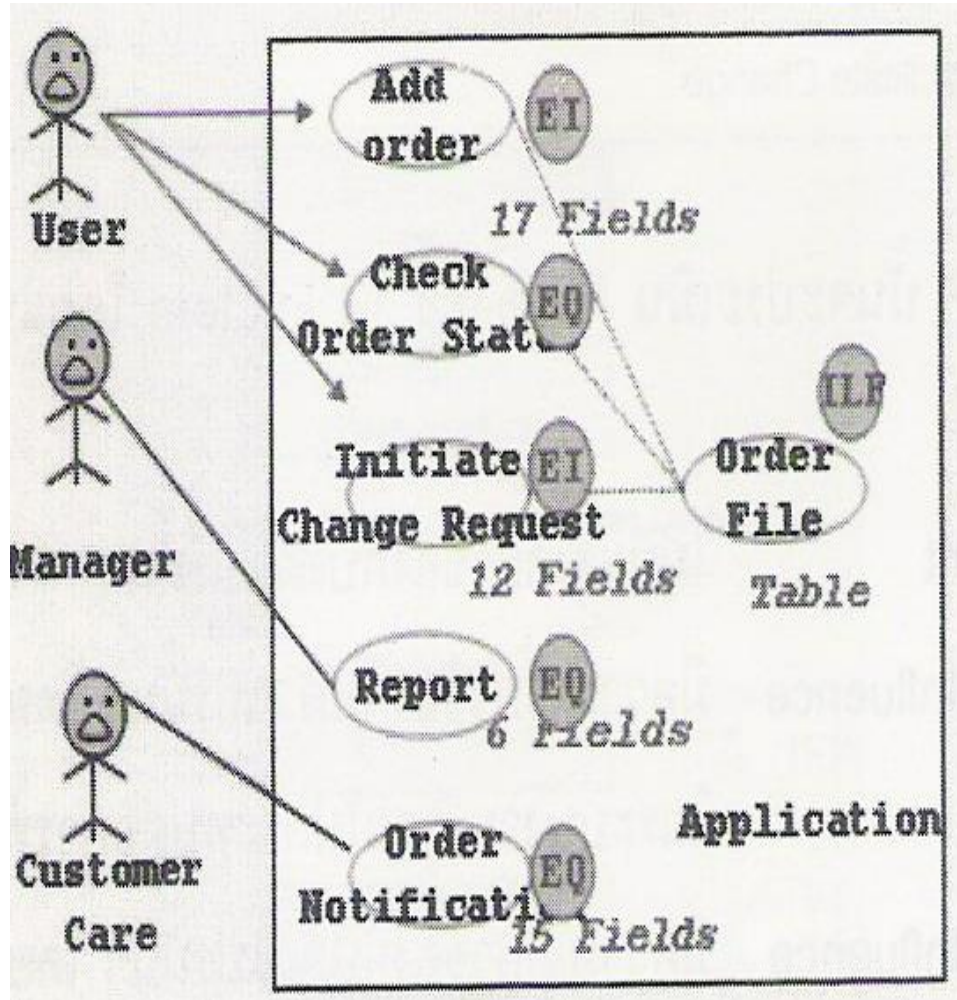
DI : Degree of Influence

ตารางเปรียบเทียบค่า FP เพื่อแปลงไปเป็น LOC

Language	SLOC/FP
Access	38
C	128
C++	55
HTML 3.0	15
Java	53
Perl	27
Visual C++	34

(From Capers Jones, 1996)

ตัวอย่างการคำนวณค่าฟังก์ชันพอยต์



- จาก Use case Diagram ดังรูป จะทำการแยกประเภทของ use case ตามฟังก์ชันพอยต์

ตัวอย่าง

Item	Type	DETs	RETs/ FTRs	Complexity	Value
Check Order Status	EQ	17	1	Low	1X 3
Add Order	EI	17	1	Average	1X 4
Initiate Change Request	EI	12	1	Low	1 x3
Order Notification	EQ	15	1	Low	1 x 3
Report	EQ	6	1	Low	1 x 3
Order File	ILF	17	1	Low	1 x 7
Summary Unadjusted Function Point: UFP					23 FP

ตัวอย่าง

- ทำการเปรียบเทียบค่าของ Value Adjustment Factors : VAF

1. Data communication	5
2. Distributed data processing	2
3. Performance	3
4. Heavily use configuration	0
5. Transaction rate	0
6. Online data entry	4
7. End user efficiency	2
8. Online Update	4
9. Complex processing	0
10. Reusability	3
11. Installation ease	0
12. Operational ease	0
13. Multiple sites	0
14. Facilitate Change	0
Total Degree Influence (TDI)	17

ตัวอย่าง

$$\text{UFP} = 23$$

$$\begin{aligned}\text{VAF} &= 0.65 + [0.01 \times 17] \\ &= 0.82\end{aligned}$$

$$\begin{aligned}\text{FP} &= \text{UFP} \times \text{VAF} \\ &= 23 \times 0.82 \\ &= 18.86 \text{ FP}\end{aligned}$$

ถ้าหากจัดทำซอฟต์แวร์โดยใช้ภาษาจาวา จะได้ค่า LOC

$$= 18.86 \times 53 = 999.58 \quad \sim 1000 \text{ LOC}$$

การประมาณการผลิต

- Productivity : ประสิทธิภาพในการผลิตงาน

$$\text{Productivity} = \frac{\text{Output Size (LOC or Function Point)}}{\text{Effort (Man-Month)}}$$

- ตัวอย่าง ถ้าโค้ดของโปรแกรมมีทั้งหมด 6,000 บรรทัด ใช้เวลาในการผลิต 8 เดือน

$$\begin{aligned}\text{Productivity} &= 6,000 / 8 \\ &= 750 \quad \text{บรรทัดต่อเดือน}\end{aligned}$$

แบบฝึกหัด จงคำนวณหาค่า FP และ LOC

Item	Type	DETs	RETs/FTRs	Complexity	Value
1	EI	20	2	?	?
2	EQ	15	1	?	?
3	ILF	10	2	?	?
4	EI	20	3	?	?
5	EIF	5	1	?	?
6	EQ	12	2	?	?
รวม UFP					?

Total Degree of Influence = 20

คำนวณค่า LOC จากการใช้ ภาษา C++